GSoC 2026 Proposal

Agentic Dora Agent-Driven Robot Control

Dora-rs | 30.03.2026 | Medium Project

Mentors

Yu Chen
chyyyuuu@gmail.com

Bob Ding
ding@nupytlot.com

Gary Ding

Dartmouth College, Hanover, NH, USA

Name and Contact Information

- **Name:** Gary Ding
- **Email:** gary.d.ding.27@dartmouth.edu
- **GitHub:** 23garyd (https://github.com/23garyd)
- **Location:** Hanover, NH, USA
- **Timezone:** UTC-4 (Eastern)

Synopsis

Robots today are controlled through rigid, pre-programmed pipelines. A pick-and-place task requires a developer to manually wire sensor nodes, planner nodes, trajectory executors, and gripper controllers in a fixed YAML dataflow — and if anything fails mid-execution, the entire pipeline stops with no way to recover. What if you could simply tell a robot “pick up the red ball and place it on the green plate” and have an AI agent figure out the rest?

This project builds a bridge between the Octos agent framework and the Dora dataflow system, enabling natural-language-driven robot control in MuJoCo simulation. The agent decomposes high-level commands into structured multi-step pipelines, translates each step into dora node interactions (planning, IK solving, trajectory execution, gripper control), and handles errors adaptively — all without requiring the user to understand the underlying dataflow topology.

A working prototype already demonstrates the core concept: a complete 18-step pick-and-place pipeline where an LLM agent grasps a red ball and places it on a green plate using a

UR5e arm with Robotiq 2F-85 gripper. This proposal describes the path from prototype to production-quality deliverable.

What It Means to Accomplish

- **Simulation Environment:** A reproducible MuJoCo simulation with a UR5e 6-DOF arm + Robotiq 2F-85 gripper, red ball, and green plate — all exposed as dora nodes with sensor outputs (joint positions, velocities) and actuator inputs (trajectory commands, gripper control).
- **Agent Bridge Node:** A dora node that embeds the Octos agent pattern (LLM provider, tool registry, skill loader, pipeline executor, safety hooks) and translates agent decisions into dora dataflow messages.
- **Agent-Callable Robot Tools:** Six tools (dora_read, dora_send, dora_call, dora_move, dora_wait, dora_list) that expose the full dora dataflow to the agent as structured function calls with JSON schemas.
- **End-to-End Demo:** A runnable demo where a user types "pick up the red ball and place it on the green plate" and the agent executes an 18-step pipeline — reading state, planning motions, descending to grasp, lifting, moving to the plate, placing, and returning home.
- **Example Dataflow:** A complete dora YAML configuration (octos_agent_demo.yml) with 7 nodes demonstrating the full pipeline from natural language command to physical robot action.
- **Documentation:** Setup instructions, architecture overview, tool API reference, skill authoring guide, and instructions for extending the system to new robots and tasks.

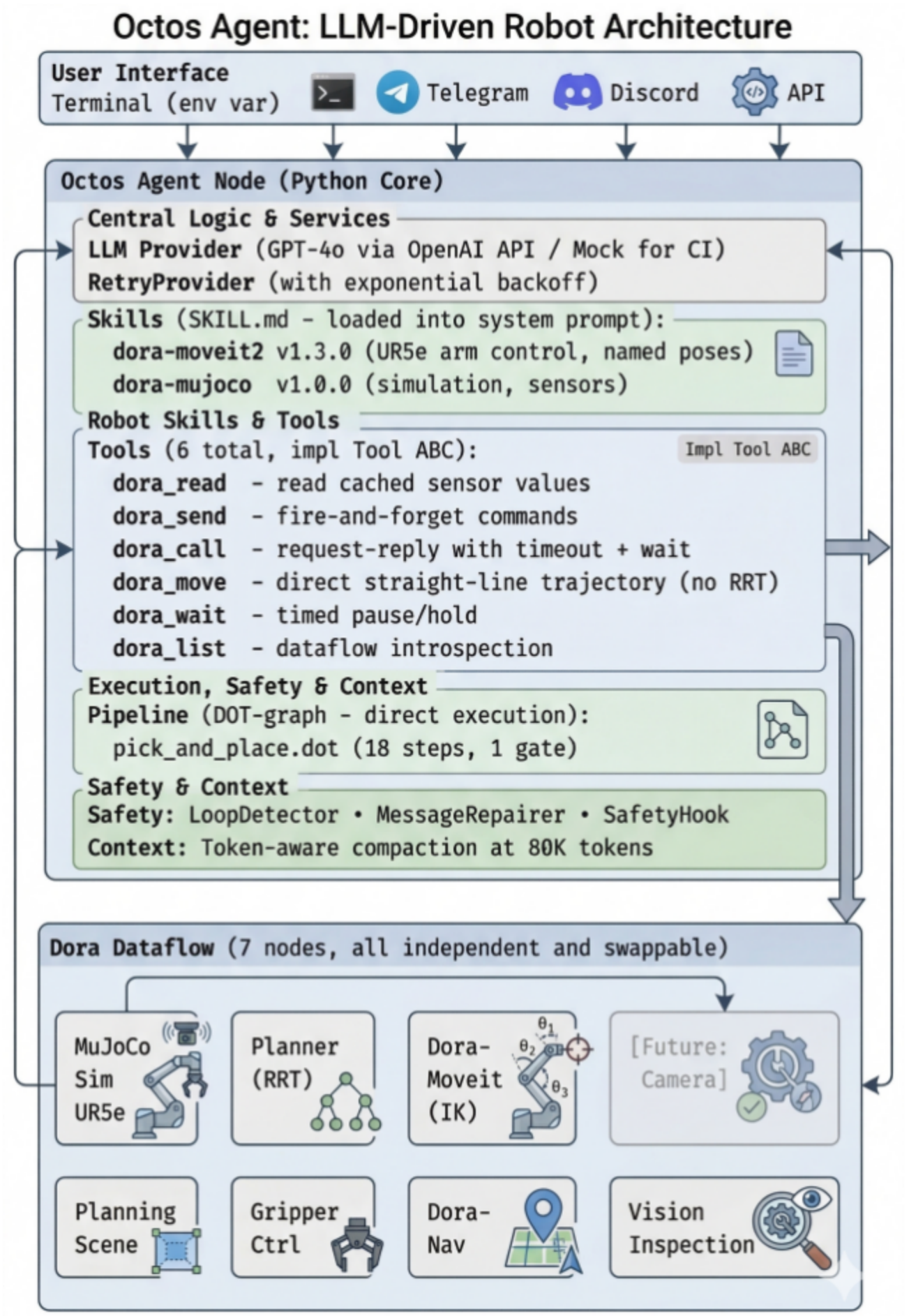
How Will It Benefit Dora?

- **New Interaction Paradigm:** Dora currently requires users to understand YAML dataflow topology to control robots. An agent bridge node enables natural language control — dramatically lowering the barrier to entry for robotics developers.
- **Adaptive Execution:** Static dataflows have no error recovery. If a motion plan fails, the pipeline stops. An agent can re-plan with different approach angles, retry with adjusted parameters, or ask the user for guidance.
- **Skill Portability:** Robot knowledge lives in SKILL.md files (named poses, node APIs, workflows), not hardcoded in node logic. Users can add new robots by writing a skill file rather than modifying source code.
- **Provider Flexibility:** The same dataflow works with GPT-4o, Claude, Ollama, or a deterministic mock provider. Researchers can test with mock providers (no API keys) and deploy with production LLMs.
- **Community Growth:** An agentic demo that works out of the box lowers the barrier for new contributors to the Dora ecosystem. Natural language control is a compelling demo that attracts developers from the broader AI/robotics community.

- **Ecosystem Integration:** The Octos agent pattern (tools, skills, pipelines) provides a reusable template for building agent-driven dora nodes beyond robotics — logistics, industrial automation, drone control.

Architecture & Design

Three-Layer Architecture



Why Octos

The project requires an agent framework that can orchestrate multi-step robot tasks reliably. Three open-source frameworks were evaluated:

Dimension	Octos	ZeroClaw	OpenClaw
Language	Rust	Rust	Python
Pipeline orchestration	DOT-graph with fan-out, gates, checkpoints	None	None
Provider failover	3-layer (retry → chain → adaptive)	Single retry	Basic retry
Loop detection	Built-in	None	None
Message repair	Fixes malformed LLM JSON	None	None
Tool management	LRU lifecycle, base-tool pinning	Manual registration	Manual registration
Context compaction	Token-aware trimming	None	None
Safety hooks	Lifecycle hooks + circuit breaker	Lifecycle hooks	None
Skills format	SKILL.md	SKILL.md	Custom

Octos was chosen because pipeline orchestration and direct step execution solved the most critical problem in agent-driven robot control: ensuring every step of a multi-step task executes in the correct order. **This capability does not exist in ZeroClaw or OpenClaw and would need to be built from scratch.**

In practice, this matters enormously. A pick-and-place task has 18 deterministic steps. Without pipelines, the LLM must hold the entire plan in working memory — one reasoning mistake (skipping a step, wrong order) breaks the physical chain. With Octos DOT-graph pipelines, each step is explicitly defined and directly executed.

Deliverables

- **MuJoCo Simulation Environment:** UR5e + Robotiq 2F-85 gripper in MuJoCo with physics-based grasping (friction-based, no weld constraints). Scene objects (red ball, green plate) added dynamically via planning scene commands. Sensor outputs and actuator inputs exposed as dora node I/O.
- **Agent Bridge Node (octos_bridge):** Python dora node implementing the Octos agent pattern — LlmProvider abstraction, ToolRegistry with LRU lifecycle and base-tool pinning, SKILL.md skill loader, DOT-graph pipeline parser and executor, safety infrastructure

(LoopDetector, MessageRepairer, SafetyHook, CircuitBreaker), and token-aware context compaction.
- **Six Agent-Callable Tools:** dora_read (cached sensor values), dora_send (fire-and-forget), dora_call (request-reply with 2-phase wait), dora_move (straight-line trajectory), dora_wait (timed pause), dora_list (dataflow introspection).
- **Pick-and-Place Pipeline:** 18-step DOT-graph pipeline with direct execution (parsed from step labels, bypassing LLM for deterministic steps) and 1 human-in-the-loop gate.
- **Mock Provider for Testing:** Deterministic scripted provider (OCTOS_PROVIDER=mock) that replays the full pick-and-place task without requiring any external API keys.
- **Comprehensive Tests:** 51+ unit tests (design conformance), E2E test harness with JSON fixture replay, and integration tests for all 6 tools.
- **Documentation:** Architecture overview, setup guide, tool API reference, skill authoring guide, pipeline authoring guide, and extension instructions.
- **Example Dataflow Configuration:** Complete octos_agent_demo.yml with 7 nodes: mujoco_sim, planner, ik_solver, trajectory_executor, planning_scene, gripper_controller, octos_bridge.

Additional Deliverables Based on Progress

If development progresses ahead of schedule, the following additional deliverables may be implemented:

- **Rust-Native Bridge:** Port the Python agent bridge to Rust using the adora API for in-process tool calls, zero-copy Arrow messaging, and lower latency.
- **Camera Node + Vision-Based Grasping:** Add a MuJoCo camera sensor node with RGB output. Replace hardcoded positions with vision-based object detection.
- **Telegram/Discord Channel Integration:** Connect the Octos messaging channel to the agent bridge for remote robot control.
- **Multi-Object Sorting Demo:** Extend the pipeline to sort multiple objects by color using parallel DOT-graph fan-out.
- **Episode Memory:** Persistent cross-session memory so the agent remembers past strategies.
- **Mobile Base + Arm Demo:** Integrate the mobile robot with arm from dora-moveit2 examples for navigation + manipulation tasks.

Milestones

Phase I

- MuJoCo simulation environment with UR5e + gripper (milestone 1)
- Agent bridge node with tool registry and LLM provider (milestone 2)

Phase II

- Six agent-callable tools and pick-and-place pipeline (milestone 3)
- Safety infrastructure and direct pipeline execution (milestone 4)

Phase III

- Testing framework and comprehensive tests (milestone 5)
- Documentation, example dataflow, and demo polish (milestone 6)

Schedule

Community Bonding Period — Pre-Development Phase (May 8 – June 1)

- **Dora Codebase Familiarization:** Set up development environment, explore Dora node API (Python and Rust), understand the dataflow YAML format, and study existing examples.
- **Octos Architecture Study:** Understand the Octos agent pattern — ToolRegistry, LlmProvider, SKILL.md format, DOT-graph pipelines, safety hooks.
- **MuJoCo + dora-moveit2 Deep Dive:** Run the Dora-moveit demo, understand the planner (RRT-Connect), IK solver (TracIK), trajectory executor (quintic spline), and planning scene nodes.
- **Establish Communication:** Set up weekly check-ins with mentors, join the Dora Discord, and align on technical requirements.

Phase I: Foundation (June 1 – July 14)

1. MuJoCo Simulation Environment (week 1–3)

- UR5e 6-DOF arm + Robotiq 2F-85 gripper model with joint configs
- Physics-based grasping: friction parameters, force range, contact dynamics tuned for grip
- Scene objects (red ball, green plate) with freejoint dynamics for free movement
- Gripper controller node: gripper_command input → gripper_ctrl output
- Dataflow wiring: mujoco_sim ↔ trajectory_executor ↔ planner ↔ ik_solver ↔ planning_scene ↔ gripper_controller

Outcome: Working simulation where arm moves to commanded joint positions and gripper physically grasps objects

2. Agent Bridge Node with Tool Registry (week 4–6)

- Octos agent pattern: Agent class with configurable max_iterations, max_timeout, tool_timeout
- LlmProvider abstraction: OpenAIProvider (GPT-4o, retry, token) and MockProvider
- ToolRegistry: registration, discovery, invocation with JSON schemas
- SKILL.md loader: YAML parsing, system prompt injection

Outcome: Agent bridge node that receives natural language commands, invokes LLM, and returns responses via dora dataflow

Phase II: Tools and Pipeline (July 15 – August 11)

3. Six Agent-Callable Tools and Pipeline (week 7–9)

- dora_read: cache lookup with age_ms tracking
- dora_send: JSON → Arrow serialization, fire-and-forget
- dora_call: 2-phase wait — Phase 1 (plan_status response) + Phase 2 (execution complete)
- dora_move: direct trajectory for straight-line motion
- dora_wait: timed pause with event draining
- dora_list: static node/IO map for dataflow
- Plan request auto-normalization: missing start injection, named pose resolution
- DOT-graph pipeline parser: topological ordering, step descriptions, gate nodes

Outcome: Agent can execute the full 18-step pick-and-place pipeline through tool calls

4. Safety Infrastructure and Direct Execution (week 10–11)

- LoopDetector: breaks infinite tool call loops
- MessageRepairer: fixes malformed JSON from LLM
- SafetyHook: validates joint limits before plan execution
- CircuitBreaker: auto-disables after N consecutive failures
- Context compaction: token-aware message trimming at 80K tokens
- Direct pipeline execution: parse tool calls from DOT step labels, execute deterministically pattern: RRT planner for lateral moves, dora_move for vertical descent/lift

Outcome: Reliable, safe, deterministic pipeline execution with fallback to LLM for unparseable steps

Phase III: Quality and Documentation (August 12 – August 25)

5. Testing Framework and Comprehensive Tests (week 12)

- Design conformance test suite: 51+ unit tests covering all components (no MuJoCo, dora, or API keys required)
- E2E test harness: JSON fixture replay with mock provider
- Integration tests: all 6 tools against mock dora node
- Mock provider validation: scripted pick-and-place replays deterministically

Outcome: Full test suite runnable in CI without hardware, simulation, or API keys

6. Documentation and Demo Polish (week 13)

- Architecture overview with dataflow topology diagram
- Setup guide: pip install + dora start in under 5 minutes
- Tool API reference: JSON schemas, usage examples, error handling
- Skill authoring guide: how to write SKILL.md for new robots
- Pipeline authoring guide: DOT-graph format, gate nodes, direct execution syntax
- Extension guide: adding new tools, providers, and robots
- Demo video/GIF: full pick-and-place from natural language command to completion

Outcome: Fully documented framework ready for community adoption

Availability

Throughout the GSoC program period (May 8th to September 1st), I will be on summer break from Dartmouth College with no academic obligations. I will dedicate 35–40 hours per week to the project for the entire duration, with full availability on weekends for mentor meetings, collaborative sessions, or intensive development sprints.

Publishing the Code

Pull Request Strategy:

- I plan to submit at least one pull request (PR) for each milestone achieved throughout the project.
- PRs will target the Dora/dora or Dora/adora repository (as directed by mentors) on a feature branch.
- Each PR will include tests, documentation, and a demo showing the new functionality.
- Once all milestones are completed, the feature branch will be merged into main.

Adherence to Best Practices:

- All code will follow existing Dora conventions.
- Commit messages will use conventional commit format (feat:, fix:, docs:, test:).
- Mock provider ensures all demos and tests run without external API keys.

Responsiveness to Feedback:

- I commit to responding to PR review comments within 24 hours.
- I will iterate on contributions to align with project standards and mentor feedback.

Past Contributions

Dora/dora-moveit2

PR: UR5e + Robotiq 2F-85 Pick-and-Place Demo — Merged, March 25, 2026

Introduced a complete UR5e robotic arm model using MuJoCo with official Menagerie meshes, integrated a Robotiq 2F-85 gripper featuring tendon-coupled 4-bar linkage actuation, and implemented PD general actuators. This PR added the robot model, configuration, and dataflow YAML that serves as the foundation for the agent-driven demo in this proposal.

- UR5e MuJoCo model with FK-verified joint configurations for 9 named poses
- Robotiq 2F-85 gripper with friction-based physics grasping (no weld constraints)
- Robot configuration class implementing the RobotConfig protocol
- Dataflow YAML (ur5e_example_mujoco.yml) wiring all nodes

octos-org/octos

Issue #166: Doc URL Broken — Open, March 29, 2026

Reported a 404 error for the Chinese documentation link, demonstrating engagement with the Octos project.

dora-octos-bridge (Pre-GSoC Prototype)

Built a complete working prototype (v0.2.10, 15 releases) of the agent-dora bridge as pre-GSoC preparation. This prototype demonstrates the feasibility of the approach:

- 6 agent-callable tools with 2-phase execution wait
- DOT-graph pipeline with direct execution
- Mock provider for deterministic testing without API keys

OpenCV (GSoC 2022)

Python and Java Libraries for OAK-D — GSoC 2022 Intern

Created Python and Java libraries for OAK-D depth camera person detection as part of the OpenCV IronOak project. Implemented threading optimization, ran tests under Ubuntu, and gained proficiency in open-source contribution workflows.

prasannavk/python-ironoak

PR: Person Detector — Merged, June 7, 2022

Added a person detector wrapper class with test functionality for the OAK-D depth camera Python library.

Work Tracking

I will track progress using GitHub Projects with a Kanban board, allowing mentors to monitor work in real time. Each milestone will have a tracking issue with checkboxes for sub-tasks. Weekly status updates will be posted to the Dora Discord channel.

Creation of a Blog

To document my GSoC journey, I will maintain a blog showcasing progress, challenges, and key learnings. Posts will cover technical deep-dives (e.g., "Why LLMs Skip Steps in Multi-Step Robot Control" and "RRT Paths Are Not Straight Lines") alongside weekly progress summaries.

Final Presentation

At the end of the project, I will present my work to the Dora community via a recorded demo and a presentation on the Dora Discord or a community call. The demo will show the full pick-and-place pipeline from natural language command to robot action in MuJoCo, including error recovery and mock provider testing.

About Me

Education

University: Dartmouth College, Hanover, NH, USA

Major: Environmental Earth Science modified with Computer Science (B.A.)

Graduation Year: 2027 | GPA: 3.94

Why This Project Is Important to Me

I have been working with robot arms since high school. At FutureDial, I spent three summers programming a 6-axis robot arm for assembly line applications — path planning, trajectory execution, transformation matrices, and digital I/O. I reproduced Stanford's UMI project to apply machine learning to a phone pick-and-place task in a warehouse, replacing rule-based robot control with learned policies. More recently, I assembled a Koch 1.1 robot arm and reproduced Stanford's ALOHA project for imitation learning.

When I discovered Dora, I saw an opportunity to combine my robotics experience with my interest in AI agents. The idea that a robot could be controlled through natural language — that an agent could decompose "pick up the red ball" into 18 discrete steps and execute them reliably — felt like the natural evolution of the manual pipeline programming I had been doing for years.

Past Experience

Robotics: At FutureDial, I programmed a 6-axis robot arm using Python and RoboDK simulation software for assembly line applications. I applied remote socket programming, transformation matrices, and digital I/O for path and trajectory planning. I created a vision pipeline using OpenCV for QR code detection and dynamic positioning. I reproduced Stanford's UMI project for ML-based pick-and-place and Stanford's ALOHA project with a Koch 1.1 robot arm for imitation learning.

Open Source: I was a GSoC 2022 intern with OpenCV, creating Python and Java libraries for OAK-D depth camera person detection. I developed deep learning perception modules for person detection, fire detection, and robot planning modules for obstacle avoidance using ROS and 2D LiDAR. I designed and built an autonomous farm robot using an OAK-D depth camera, ArduPilot, Nvidia Jetson, and GNSS/RTK systems.

AI/ML: At Thayer School of Engineering (Dartmouth), I created machine learning models for wildfire resource allocation using geospatial data and weather conditions. I have hands-on experience with OpenCV, deep learning inference pipelines, and LLM agent frameworks (Octos, OpenAI API).

Languages & Tools: Python, Rust, C, Java, ROS, MuJoCo, OpenCV, Dora, ArduPilot, Linux, Git.

Resources

- **Dora Repository:** https://github.com/Dora/adora
- **Dora Docs:** https://Dora.ai/docs
- **Dora Discussions:** https://github.com/orgs/Dora/discussions
- **Dora Discord:** https://discord.com/channels/1146393916472561734/1346181173277229056
- **Octos:** https://github.com/octos-org/octos
- **dora-moveit2:** https://github.com/Dora/dora-moveit2
- **python-ironoak (GSoC 2022):** https://github.com/prasannavk/python-ironoak/pull/4/commits
- **java-ironoak (GSoC 2022):** https://github.com/23garyd/java-ironoak/commits/main
- **GSoC 2022 Report:** https://23garyd.github.io/tech/gsoc-2022-report.html